Instituto Politécnico Nacional
Escuela Superior de Cómputo
Algoritmos Bioinspirados | Genetic Algorithms
6 Particle Swarm Optimization
Malagón Baeza Alan Adrián
Rosas Trigueros Jorge Luis
23/10/2023
10/11/2023

Theoretical Framework

Optimization is a fundamental field in computer science and mathematics used to find the best value of an objective function, known as the optimum, within a search space. In optimization, the goal is to find the solution that minimizes or maximizes an objective function, subject to certain constraints or conditions.

There are two main types of optimization: local optimization and global optimization. Local optimization focuses on finding the optimum in a specific region of the search space, while global optimization aims to find the global optimum, i.e., the best value across the entire search space.

The Rosenbrock function is a common test function used to evaluate optimization algorithms. In three dimensions, it is defined by the following formula:

$$f(x, y, z) = (a - x)^2 + b * (y - x^2)^2 + c * (z - y^2)^2$$

The Rastrigin function is another test function used for evaluating optimization algorithms. In four dimensions, it is defined by the following formula:

$$f(x, y, z, w) = A * n + \sum_{i=1}^{n} \left[x_i^2 - A * \cos(2\pi x_i)\right]$$

PSO is an optimization algorithm inspired by the behavior of birds in a swarm. Each "particle" in the swarm represents a candidate solution in the search space. The algorithm iteratively seeks to improve solutions by moving particles through the space based on their personal and group experiences. This is achieved by updating the velocity and position of each particle in each iteration.

To apply PSO to the Rosenbrock and Rastrigin functions, the algorithm's parameters are configured, including swarm size, maximum velocity, inertia, cognitive and social influences, among others. The goal is to use PSO to find the variable values that minimize these functions in their respective dimensions.

Material and equipment

The computational environment utilized for this project is Google Colab (Google Colaboratory). Google Colab is a cloud-based Python development platform that offers free access to a virtual environment for running Python code, executing data analysis, and performing machine learning tasks.

Notebook Setup

- Execution environment type: Python 3
- Hardware accelerator: CPU

Operating System:

- Distributor ID: Ubuntu
- Description: Ubuntu 22.04.2 LTS (Long Term Support)
- Release: 22.04
- Codename: jammy

CPU Information:

- Architecture: x86_64
- CPU op-mode(s): 32-bit, 64-bit
- CPU(s): 2
- Vendor ID: GenuineIntel
    - Model name: Intel(R) Xeon(R) CPU @ 2.20GHz

Practice development

Use PSO to optimize:

- Rosenbrock's function in 3 dimensions.

This code implements a Particle Swarm Optimization (PSO) algorithm to optimize a 3D function, specifically the Rosenbrock function. PSO is a population-based optimization technique, and in this code, the search space is defined by lower and upper limits. The algorithm initializes a group of particles with random positions and velocities in the search space. It then iteratively updates the particle positions and velocities based on their past performance and the best solutions found so far. The key PSO parameters like the inertia weight (weight), cognitive factor (C1), and social factor (C2) are used to control the balance between exploration and exploitation. The algorithm tracks both local best positions for each particle and the global best position among all particles. After each iteration, the code updates and prints the count, the best particle's position, and the value of the Rosenbrock function at the global best position, helping to visualize the optimization process using an "Next Iteration" button.

```
# Import necessary libraries
import ipywidgets as widgets
from IPython import display as display
import matplotlib.pyplot as plt
import numpy as np

def create_button():
    # Create the "Next Iteration" button widget
    button = widgets.Button(
        description='Next Iteration',
        disabled=False,
        button_style='',
        tooltip='Next Iteration',
        icon='check'
    )
    return button

# Define the search space limits
lower_limit = -20
upper_limit = 20

# Number of particles and dimensions for the 3D optimization problem
n_particles = 20
n_dimensions = 3

# Define Rosenbrock's function
```

```
def rosenbrock(x, y, z):
    a = 1.0
    b = 100.0
    c = 100.0
    return (a - x)**2 + b * (y - x**2)**2 + c * (z - y**2)**2

# Initialize particle positions and velocities in the 3D space
X = lower_limit + (upper_limit - lower_limit) *
np.random.rand(n_particles, n_dimensions)
V = -(upper_limit - lower_limit) / 2 + (upper_limit - lower_limit) *
np.random.rand(n_particles, n_dimensions)

# Initialize global and local fitness values
fitness_gbest = np.inf
fitness_lbest = np.full(n_particles, np.inf)

# Initialize global and local best positions
X_lbest = np.copy(X)
X_gbest = X_lbest[0].copy()

# Initialize fitness values for the current particle positions
fitness_X = np.zeros(n_particles)

count = 0

# Define PSO parameters
weight = 0.5
C1 = 0.3
C2 = 0.2

def iteration(b):
    global count
    global weight, C1, C2
    global X, X_lbest, X_gbest, V

    display.clear_output(wait=True)
    display.display(button)
    count += 1

    # Update the particle velocities and positions
    for i in range(n_particles):
        for j in range(n_dimensions):
            R1 = np.random.rand()
            R2 = np.random.rand()
```

```
            V[i][j] = (weight * V[i][j] + C1 * R1 * (X_lbest[i][j] -
X[i][j]) + C2 * R2 * (X_gbest[j] - X[i][j]))
            X[i][j] = X[i][j] + V[i][j]

        # Calculate the fitness of the new position
        fitness_X[i] = rosenbrock(X[i][0], X[i][1], X[i][2])

        # Update local best position if necessary
        if fitness_X[i] < fitness_lbest[i]:
            X_lbest[i] = X[i].copy()
            fitness_lbest[i] = fitness_X[i]

        # Update global best position if necessary
        if fitness_X[i] < rosenbrock(X_gbest[0], X_gbest[1],
X_gbest[2]):
            X_gbest = X_lbest[i].copy()

    print(count, "Best particle in:", X_gbest, " gbest: ",
rosenbrock(X_gbest[0], X_gbest[1], X_gbest[2]))

# Create the "Next Iteration" button
button = create_button()
button.on_click(iteration)
display.display(button)
```

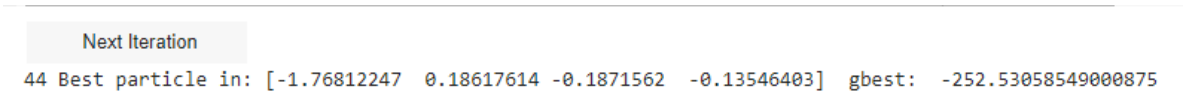

Figure 1. Result for Rosenbrock

- Rastrigin's function in 4 dimensions.

This code is very similar to the previous one, but it is adapted for a different optimization problem. Here, it optimizes a 4D function known as the Rastrigin function within the specified search space. The Rastrigin function is a well-known multimodal optimization problem. It initializes a population of particles in a 4D space and iteratively updates their positions and velocities using the Particle Swarm Optimization (PSO) algorithm. The PSO parameters, such as the inertia weight (weight), cognitive factor (C1), and social factor (C2), control the balance between exploration and exploitation. The code keeps track of both local best positions for each particle and the global best position among all particles, printing the count, the best particle's position, and the value of the Rastrigin function at the global best position after each iteration. You can visualize the optimization process using the "Next Iteration" button in Jupyter Notebook.

```
# Import necessary libraries
import ipywidgets as widgets
from IPython import display as display
import matplotlib.pyplot as plt
import numpy as np

def create_button():
    # Create the "Next Iteration" button widget
    button = widgets.Button(
        description='Next Iteration',
        disabled=False,
        button_style='',
        tooltip='Next Iteration',
        icon='check'
    )
    return button

# Define the search space limits
lower_limit = -5.12
upper_limit = 5.12

# Number of particles and dimensions for the 4D optimization problem
n_particles = 20
n_dimensions = 4

# Define Rastrigin's function
A, B, C, D = 10, 10, 10, 10

def rastrigin(x1, x2, x3, x4):
```

```
    return A * (4 - 2.1 * (x1**2) + (x1**4) / 3) + x1 * x2 + B *
(x2**2 - 4.86) + (x2**4) / 6 + x2 * x3 + C * (x3**2 - 8.7) + (x3**4) /
2 + x3 * x4 + D * (x4**2 - 12.44)

# Initialize particle positions and velocities in the 4D space
X = lower_limit + (upper_limit - lower_limit) *
np.random.rand(n_particles, n_dimensions)
V = -(upper_limit - lower_limit) / 2 + (upper_limit - lower_limit) *
np.random.rand(n_particles, n_dimensions)

# Initialize global and local fitness values
fitness_gbest = np.inf
fitness_lbest = np.full(n_particles, np.inf)

# Initialize global and local best positions
X_lbest = np.copy(X)
X_gbest = X_lbest[0].copy()

# Initialize fitness values for the current particle positions
fitness_X = np.zeros(n_particles)

count = 0

# Define PSO parameters
weight = 0.5
C1 = 0.3
C2 = 0.2

def iteration(b):
    global count
    global weight, C1, C2
    global X, X_lbest, X_gbest, V

    display.clear_output(wait=True)
    display.display(button)
    count += 1

    # Update the particle velocities and positions
    for i in range(n_particles):
        for j in range(n_dimensions):
            R1 = np.random.rand()
            R2 = np.random.rand()
            V[i][j] = (weight * V[i][j] + C1 * R1 * (X_lbest[i][j] -
X[i][j]) + C2 * R2 * (X_gbest[j] - X[i][j]))
            X[i][j] = X[i][j] + V[i][j]
```

```
        # Calculate the fitness of the new position
        fitness_X[i] = rastrigin(X[i][0], X[i][1], X[i][2], X[i][3])

        # Update local best position if necessary
        if fitness_X[i] < fitness_lbest[i]:
            X_lbest[i] = X[i].copy()
            fitness_lbest[i] = fitness_X[i]

        # Update global best position if necessary
        if fitness_X[i] < rastrigin(X_gbest[0], X_gbest[1],
X_gbest[2], X_gbest[3]):
            X_gbest = X_lbest[i].copy()

    print(count, "Best particle in:", X_gbest, " gbest: ",
rastrigin(X_gbest[0], X_gbest[1], X_gbest[2], X_gbest[3]))

# Create the "Next Iteration" button
button = create_button()
button.on_click(iteration)
display.display(button)
```

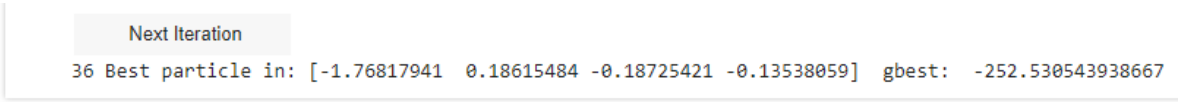

Figure 2. Result for Rastrigin

Conclusions and recommendations

Both codes demonstrate effective implementations of the PSO algorithm, showcasing its versatility in optimizing different functions within specified search spaces. The utilization of interactive widgets in platform like Google Colab allows for an engaging and step-by-step exploration of the optimization process, which can greatly aid in understanding the algorithm's behavior.

To enhance the educational value of these codes, it is recommended to include more comprehensive comments and explanations, especially for users who may be new to PSO or programming. Providing context about the specific optimization problems being solved and their characteristics would further aid users in grasping the nuances of the codes and the functions they optimize. Additionally, visual aids such as fitness landscapes or diagrams could be incorporated to illustrate how PSO seeks optimal solutions in complex spaces.

References

- Google Colaboratory. (n.d.). https://colab.research.google.com/
- Python Software Foundation. (2023). Python Documentation. https://docs.python.org/3
- Matplotlib documentation — Matplotlib 3.7.2 documentation. (n.d.). https://matplotlib.org/stable/index.html
- NumPy documentation — NumPy v1.25 Manual. (n.d.). https://numpy.org/doc/stable/